Test strategy: covering the critical systems

Status: investigation. This document inventories the current tests, identifies the critical systems with the weakest property, snapshot, or integration coverage, and proposes a prioritized batch. It does not change the suite.

Method

A **critical system** is one where a defect is silent, irreversible, or crosses a trust boundary: the kernel and its ABI/image, lowering and the abstraction, the parsers, the pattern matcher, verification and refinement, persistence, the actor scheduler, memory and epochs, the writer, the C adapter, and the prose front-end. A test is scored by the failure it would catch, not by line coverage.

The three test types have distinct jobs:

- **Properties** falsify an invariant over generated inputs. Prefer them where the input space is large or the invariant is structural. They are the strongest anti-regression device because they can falsify a claim, not merely confirm an example.
- **Snapshots** pin a canonical artifact where an unintended shape change is the risk: emitted F#, SMT obligations, portable JSON, lowered IR, and rendered diagnostics.
- **Integration tests** prove the seams: prose to evaluator, kernel to C adapter, persistence across a process, the scheduler under interleaving, and the published payload.

What exists

The suite has three runners:

- the example suite (`development/scale/tests.fs`): 1530 checks, 1488 examples and 42 Hedgehog properties;
- the granular suite (`development/scale/granular-tests.fs`): 33 cases, 11 Verify snapshots, 13 integrations, and 9 JIT-only writer checks;
- the differential corpus (`development/scale/differential.fs`): 18 token programs run through both the interpreter and the lowered abstraction.
- discovery: 91 tests.

The C conformance harness (`components/adapter-c/conformance/conformance.c`) is compiled in alpha and run against the NativeAOT kernel by `just kernel-aot` (D180). The same gate runs in the path-filtered CI workflow and before a release.

System	Examples	Properties	Snapshots	Integration
Kernel, ABI, image	strong	5 image properties	—	NativeAOT kernel + C conformance, alpha gate (D180)
Lowering and abstraction	18-program differential corpus	1 differential property	1 lowered-IR snapshot	—
Pattern matching	strong	—	—	—
Prose sentence parser	strong	1 generated-sentence property	—	demo loop
Prose surface	36 checks	1 forms property	—	stack seam, CLI

System	Examples	Properties	Snapshots	Integration
Verification, SMT, refinement	strong	—	10 snapshots	fixed-spec, demo
Persistence	strong	4 properties	—	mailbox queue, demo
Actors and scheduler	strong	6 properties	—	actor model
Memory and epochs	strong	4 properties	—	—
Writer and IL	9 JIT checks	—	—	granular runner
C adapter	compile check	—	—	NativeAOT kernel + native harness, alpha gate (D180)

Gaps and proposed tests

Lowering and the abstraction (highest risk)

The only oracle comparing the interpreter and the lowered abstraction was a fixed 18-program corpus. The writer, the AOT path, and the compiled subset depend on that agreement, but a newly lowered node can disagree outside the corpus.

P1 and S1 are landed (D148). P1 generates arithmetic programs and compares the interpreter with the lowered abstraction; S1 pins the differential corpus's lowered instructions. The generated subset should grow to slots, calls, `if`, `let`, `lambda`, and ground patterns; the fixed corpus no longer stands alone.

Boundary and term documents (landed batch)

The recent IPC and P1 work adds the first critical boundary batch:

- Wire malformed vectors: invalid operation, invalid body kind, missing text pointer, missing NUL terminator, text out of bounds, and a multi-segment message. The F# decoder must refuse each by name.
- Term portable malformed documents: bad form, missing field, wrong type, negative slot, negative capture, and negative arity.
- Capability forgery: an unregistered `RemoteHandleTag` tensor is not a remote handle and `send-to` refuses it.

Property coverage is still the gap. P9 should generate wire envelopes and assert encode/decode round-trip; P10 should generate bounded term trees and assert `TermPortable` round-trip; P11 should generate registered and unregistered handles and assert only registered entries become transport requests. The next integration batch should cover an unknown or retired capability, multiple senders per drain, and a text body refused by the numeric actor host.

Kernel verifier and receipt (landed batch)

P2 adds twelve checks in the example suite:

- the verifier accepts the canonical `number-add` term program and returns a 64-character SHA-256 digest;
- the receipt is canonical JSON and carries that digest;
- an empty word inside a quotation is refused at `terms[0][0]`;
- an empty closure name, a negative capture, and a non-finite number are refused;

- `scale_verify` accepts a term program, returns a receipt, refuses an invalid handle, and refuses a malformed term program with the diagnostic;
- the ABI version is 1 and the header, kernel export, and adapter target agree.

The P2 verifier is structural, not stack-typed. Stack underflow, quote input and output types, closure arity, and `apply` are P4/P3 work. The receipt is bound to the canonical program bytes, not to a host object.

Token-to-term boundary (landed batch)
P3a adds three checks in the example suite:

- numeric tokens become `LiteralNumber` and names become `Word`;
- external text splits and maps through the same token boundary;
- a non-finite token is refused, not turned into NaN.

The boundary did not group `{ ... }` into `Quote` or change the active `Program`; those are P3b/P3c. The temporary `toTokens` projection was removed in P3d.

P3b (D165) adds five checks in the example suite:

- a typed number/word program runs to 5 through `Evaluator.run`;
- bookmark rollback restores the typed program;
- the kernel bridge projects typed terms to tokens;
- the kernel bridge rehydrates the typed terms;
- `toKernelTokens` projects a quotation as braces for the surface parser.

The active `Program` is `Term list`. P3c (D166) adds three checks in the example suite:

- a quotation encodes as `[QuoteTag; documentId]`;
- the code value decodes to its canonical term document;
- a plain value is refused as code.

P3c replaces quotation text with code values and stores history as code values; the differential corpus keeps its `Case.Program` as `string list` and converts only at `runInterpreter`.

P3d (D167) removes the text fallback, `toTokens`, and the legacy token-vector history. The example suite loses the two temporary-projection checks and gains a text-is-not-code check. P3d left 1362 checks; the D168 pipeline batch adds six. D169 adds three stage-argument checks. D170 adds three explicit-partial checks. D171 adds three reverse-pipe checks, for 1377 checks (1335 examples, 42 properties). D173 (P4a) adds nine type-inference checks; P4b adds eight expression-inference checks; P4c adds five located-diagnostic checks; the primitive registry adds four checks; the located-prose path adds two; child call ranges add three; the independent-handle reset adds one; the prose let adds two, for 1530 checks (1488 examples, 42 properties).

Pipeline (landed batch)
D168 adds six checks in the example suite:

- the prose parser produces the `Pipeline` node with `LastArgument`;
- `LastArgument` desugars to a call with the value appended;
- `FirstArgument` desugars to a call with the value prepended;
- a prose pipeline runs with last-argument order;
- chained stages run left to right;
- the pipeline portable document is distinct from its desugared call.

D169 adds three checks for whitespace stage arguments. General partial application outside a pipeline, `||>`, and Elixir surface syntax remain explicit non-goals.

Explicit partial application (landed batch)
D170 adds three checks in the example suite:

- a hole in a call parses to `Partial`;
- a partial binds its hole when applied;
- the portable document keeps the `partial` tag.

The batch does not cover automatic currying, first-class word values, or tuple pipes.

Reverse pipe (landed batch)
D171 adds three checks in the example suite:

- `f <| x` parses to `ReversePipeline`;
- chained reverse pipes are right-associative;
- a reverse pipe fills a partial hole.

The reverse pipe is supported for F# package compatibility but discouraged in new code.

Prose surface
The 36 checks are examples of the two forms, the names and text rules, the nested match, the sentence sequence, the stack seam, and their refusals. The surface is new, and the comma is structural, so generated inputs matter.

P2 is landed (D148). The property generates numeric matches and asserts that the inline and line forms parse to the same tree. Extend it to spaced names, nested matches, and invalid comma placements with named refusals.

Integration, I1. Parse a `.prose` program, lower it, and run it through the evaluator; compare the value with the hand-built IR path. Parsing is not execution, and the first slice does not yet prove the seam.

Pattern matching
The matcher has strong examples but no generative oracle.

Property, P3. For a generated ground pattern and a constructed subject, compare the matcher with a small reference matcher: `Wildcard` always matches, a literal matches exactly on equality, a fresh variable binds exactly the subject, and a constructor/typed pattern matches exactly on its head and child.

Verification, SMT, and refinement
The SMT path is example- and snapshot-heavy; its strongest claim is that the compiled predicate and the solver agree. That agreement is currently pinned by chosen cases.

Property, P4. For a small generated lattice and precondition, discharge the obligation and compare the solver verdict with the compiled predicate over the lattice's universe: a `Falsified` model must falsify the predicate, a `Proved` obligation must hold on every coordinate, and `Unknown`/`Unavailable` must remain inconclusive. Bound the domain so the property stays fast.

Property, P5. Assert that an undecidable solver outcome is never counted as a pass by the refiner or the fixed-spec gate. This is a ratchet against the most dangerous failure mode.

Persistence and execution state
Property, P6. Assert that execution state is never serialized: `Program`, the program counter, ownership maps, waiters, actor handles, and monitor references do not appear in a persisted

document. Persistence already has four round-trip properties; this one pins the boundary the properties do not.

Property, P7. A document with a different schema or ABI version is refused by name. The loader examples cover this; a property over random version pairs makes the refusal total.

Actors and scheduler

Property, P8. A model-based property over random interleavings: every sent message is delivered exactly once per mailbox, per-mailbox order is preserved, a monitor fires once, and a link delivers only on an abnormal exit. The existing properties cover FIFO, budget, and handle distinctness, but not the interleaving invariants.

Memory and epochs

Property, P9. Reclamation is sound under a generated live-root set: no reachable entry is ever reclaimed, and a closed epoch's unpromoted entries are reclaimed exactly when unreachable. The current properties cover report and compaction on generated stores; this adds the live-root quantification.

Writer and IL

Property, P10. Generate lowered programs, emit them through the writer, load the emitted assembly, and compare the result with the abstraction. The writer is JIT-only, so this stays in the granular suite. It is the only way to catch a writer bug that the fixed nine checks miss.

C adapter and native boundary

Integration, I2. The native conformance harness now runs in alpha against the NativeAOT kernel (`just kernel-aot`, D180). Fuzz the version probe: every mismatched `SCALE_EXPECTED_ABI_VERSION` is refused before any other symbol resolves. Also test a null callback table and a truncated image at the ABI edge.

Priority

Ordered by risk times coverage gap:

1. **P1, lowering/interpreter differential.** The compiler and the AOT path both trust it; the current oracle is fixed.
2. **P2 and I1, prose properties and the parse-to-run seam.** The surface is new and currently example-only.
3. **P3, pattern matcher oracle.** Large input space, example-only today.
4. **P6 and P7, execution-state and version persistence.** Trust-boundary failures are silent.
5. **P8, scheduler interleavings.** Concurrency invariants are the easiest to break without an example changing.
6. **S1, lowered-IR snapshot.** Cheap and catches shape drift.
7. **P4 and P5, SMT differential and the inconclusive ratchet.** High value, but the slowest; keep the domain small.
8. **P10, writer differential.** JIT-only, granular suite.
9. **I2, native conformance at release.** Blocked on the release gate.

Constraints

- Properties run in the AOT example suite. Keep them free of `%A`, numeric `printf`, reflection, and dynamic code (section 9.5 of `AGENTS.md`).
- Snapshot tests use Verify and stay in the JIT-only granular suite, because the native payload does not carry the baselines (D23).

- The SMT properties need Z3; a missing solver must report `Unavailable` or an inconclusive property, never a silent pass.
- Bound generators so the example suite stays fast; the current 42 properties run in the same suite as 1488 examples.

Metrics

Track two numbers, not line coverage: the fraction of ledger and invariant rows with a property or ratchet behind them, and the number of critical systems whose only oracle is an example. The first should rise; the second should fall. A snapshot is not coverage of behavior; it is a change detector for an artifact.

First batch

The first batch is landed (D148): P1, P2, I1, and S1 are in the suite; I1 is the prose stack seam. The next batch is P3 (the pattern-matcher oracle), P6/P7 (execution-state and version persistence), P8 (scheduler interleavings), S1′s extension to generated lowered programs, and P4/P5 (SMT differential and the inconclusive ratchet).

Semantic test matrix

The matrix names the semantic dimensions every later feature must cover. A row without a landed check is a known gap, not an assumed pass.

Dimension	Cases	Evidence / gate
Call graphs	direct call, forward call, self recursion, mutual recursion, missing target, cycle	example suite; book checks; group runtime tests
Effect timing	host word before state change, after state change, blocked effect, failure attribution	shell and book host tests; capability gate
Capabilities	granted, denied, package required, included union, sandbox program, sandbox write path	capability gate; book checks; sandbox tests
Persistence	image round-trip, root reattachment, execution state exclusion, schema skew	image and package round-trip tests
AOT	prose, book, cast, showcase, missing payload	release-only `aot-smoke`
Fuzz and property	lowering differential, prose forms, token boundary, malformed input	Hedgehog properties; granular suite

The release gate runs the example, property, granular, integration, snapshot, and writer checks. The AOT rows run only at a release. A new semantic feature adds its case to this matrix before implementation, and the row names the test that enforces it.